

TP1 — Processamento Otimizado em Assembly ARM 64-bits

Relatório técnico

Renato Noronha Hack

26 de abril de 2026

Contents

1	Identificação	2
2	Resumo Executivo	2
3	Ambiente de Execução	2
3.1	Ferramentas utilizadas	2
3.2	Organização do projeto	2
4	Exercício 1 — Pinagem e mapeamento BCM	3
5	Exercício 2 — Endereços base do SoC	3
6	Exercício 3 — Makefile modular	4
7	Exercício 4 — Depuração com GDB	4
8	Exercício 5 — Decomposição do binário com objdump	5
9	Exercício 6 — Soma multi-palavra de 192 bits	5
10	Exercício 7 — Subtração multi-palavra de 128 bits	6
11	Exercício 8 — Conversão de inteiro para string	6
12	Exercício 9 — Conversão de string para inteiro	7
13	Exercício 10 — Conversão para double e divisão	7
14	Exercício 11 — Validação de faixa em parsing numérico	8
15	Exercício 12 — Integração final	8
16	Consolidação das Evidências	9
17	Conclusão Final	9

1 Identificação

- **Aluno:** Renato Noronha Hack
- **Disciplina:** Processamento Otimizado em Assembly ARM 64-bits
- **Trabalho:** TP1
- **Hardware-alvo:** Raspberry Pi Zero 2W

2 Resumo Executivo

Este trabalho apresenta a implementação de uma infraestrutura de desenvolvimento e de um conjunto de rotinas em Assembly ARM64 para o Raspberry Pi Zero 2W, conforme solicitado no TP1. O projeto foi organizado de forma modular, com automação de compilação via `Makefile`, validação funcional por executáveis de teste, auditoria de binários por `objdump` e inspeção de execução por `GDB`.

A estratégia adotada teve duas etapas. Primeiro, as rotinas foram desenvolvidas e validadas preliminarmente no ambiente local com `qemu-aarch64`, o que acelerou a correção de erros de lógica e contrato. Em seguida, toda a validação final foi repetida no Raspberry Pi Zero 2W, com geração de evidências reais em `evidence/raspberry/`.

O resultado final é um pacote reproduzível, composto por código-fonte, scripts auxiliares, relatório técnico, evidências de execução e um executável integrador que reúne as rotinas principais pedidas no enunciado.

3 Ambiente de Execução

3.1 Ferramentas utilizadas

Item	Ambiente de validação preliminar	Ambiente de validação final
Arquitetura	x86_64	aarch64
Execução ARM64	qemu-aarch64	Nativa no Raspberry
Compilação	aarch64-linux-gnu-gcc	gcc
Depuração	gdb-multiarch	gdb
Auditoria de binário	aarch64-linux-gnu-objdump	objdump

3.2 Organização do projeto

O projeto foi dividido nas seguintes áreas:

- `src/asm/`: rotinas Assembly principais;
- `src/c/`: harnesses de teste e driver integrador;
- `include/`: contratos e constantes compartilhadas;
- `scripts/`: automação de build, execução, captura de evidências e sessões de `GDB`;
- `docs/reference/`: documentação técnica auxiliar;
- `docs/report/`: relatório técnico;
- `evidence/raspberry/`: evidências finais executadas no hardware real.

4 Exercício 1 — Pinagem e mapeamento BCM

O primeiro exercício exigia um artefato técnico relacionando pinos físicos do header do Raspberry Pi Zero 2W com a numeração BCM usada pelo software. Esse material foi consolidado em:

- docs/reference/pinagem_bcm_raspberry_pi_zero_2w.md

A tabela contém linhas de alimentação, terra e GPIOs relevantes, deixando explícita a diferença entre o número físico do conector e o identificador BCM utilizado no código. Essa distinção é importante porque a configuração de periféricos em Assembly ocorre pelos registradores do SoC, que usam a convenção BCM, enquanto a montagem física na bancada acontece pelo número do pino.

Conclusão: a documentação produzida atende ao objetivo de conectar o “mundo físico” ao “mundo dos registradores”, reduzindo ambiguidades em futuras rotinas de GPIO e MMIO.

5 Exercício 2 — Endereços base do SoC

Foi criado o módulo `src/asm/soc_addrs.s`, responsável por definir e carregar em registradores os endereços base e offsets derivados usados como referência para MMIO:

- PERIPH_BASE = 0x3F000000
- GPIO_BASE = 0x3F200000
- UART0_BASE = 0x3F201000
- SYS_TIMER_BASE = 0x3F003000
- GPIO_GPFSEL1 = 0x3F200004

Os resultados de validação no Raspberry foram registrados em:

- evidence/raspberry/ex02/ex2_soc_addrs_output.txt
- evidence/raspberry/ex02/ex2_soc_addrs_objdump.txt

Trecho da execução:

```
PERIPH_BASE    = 0x000000003f000000 | esperado = 0x000000003f000000 | PASS
GPIO_BASE      = 0x000000003f200000 | esperado = 0x000000003f200000 | PASS
UART0_BASE     = 0x000000003f201000 | esperado = 0x000000003f201000 | PASS
SYS_TIMER_BASE = 0x000000003f003000 | esperado = 0x000000003f003000 | PASS
GPIO_GPFSEL1   = 0x000000003f200004 | esperado = 0x000000003f200004 | PASS
STATUS FINAL: PASS
```

Trecho relevante do objdump:

```
000000000400aa8 <soc_fill_addrs>:
400aa8: d2a7e001  mov  x1, #0x3f000000
400aac: d2a7e402  mov  x2, #0x3f200000
400ab4: f2a7e403  movk x3, #0x3f20, lsl #16
400abc: f2a7e004  movk x4, #0x3f00, lsl #16
400ac4: f2a7e405  movk x5, #0x3f20, lsl #16
```

Conclusão: as constantes foram materializadas corretamente no binário e a execução nativa confirmou a coerência entre os valores esperados e os valores carregados pela rotina.

6 Exercício 3 — Makefile modular

O projeto foi estruturado com um Makefile único e modular, com os alvos:

- all
- clean
- run
- test

Os artefatos são separados entre objetos C, objetos Assembly e executáveis finais, permitindo recompilação incremental. As evidências nativas deste exercício ficaram em:

- evidence/raspberry/ex03/make_clean.txt
- evidence/raspberry/ex03/make_build.txt

Trecho da compilação nativa:

```
gcc -Wall -Wextra -Werror -std=c11 -O0 -g -fno-omit-frame-pointer -fno-pie
↪ -Iinclude -c src/c/test_soc_addrs.c -o build/obj/c/test_soc_addrs.o
gcc -g -fno-pie -c src/asm/mp_add_192.s -o build/obj/asm/mp_add_192.o
gcc -g -fno-pie -c src/asm/parse_i64.s -o build/obj/asm/parse_i64.o
gcc -no-pie build/obj/c/tpl_driver.o build/obj/c/test_common.o
↪ build/obj/asm/mp_add_192.o build/obj/asm/u64_to_str.o
↪ build/obj/asm/parse_i64.o build/obj/asm/int_div_to_double.o
↪ build/obj/asm/parse_i64_range.o -lm -o build/bin/tpl_driver
```

Conclusão: o Makefile funcionou corretamente no Raspberry Pi, recompilando apenas os módulos necessários e gerando todos os executáveis previstos.

7 Exercício 4 — Depuração com GDB

Para demonstrar inspeção de registradores gerais e flags de estado, foi escolhida a rotina mp_sub_128 em um caso com borrow. A coleta foi automatizada pelo script:

- scripts/gdb/native_ex7_borrow.gdb

Esse script executa os comandos break, run, x/i \$pc, info registers e stepi em pontos críticos do fluxo. A transcrição foi salva em:

- evidence/raspberry/ex04/gdb_mp_sub_128_borrow.txt

Trecho da sessão:

```
Breakpoint 1, mp_sub_128 () at src/asm/mp_sub_128.s:9
=> 0x400d7c <mp_sub_128>: ldr x3, [x0]
x0 0x400e28
x1 0x400e38
x2 0x7fffffff170

=== Após a subtração da low word ===
=> 0x400d90 <mp_sub_128+20>: sbcs x8, x4, x6
x7 0xffffffffffffffff
cpsr 0x80200000 [ EL=0 BTYPE=0 SS N ]
```

Interpretação: após a primeira subtração, o resultado em `x7` tornou-se `0xffffffffffffffff`, evidenciando underflow da word menos significativa. No `cpsr`, a ausência da flag `C` confirma a ocorrência de borrow, o que influencia diretamente a instrução `sbc` seguinte.

Conclusão: a sessão comprova a depuração instrução por instrução, a leitura de registradores gerais e a influência efetiva das flags do `PSTATE` no comportamento da rotina.

8 Exercício 5 — Decomposição do binário com objdump

O binário escolhido para auditoria foi `build/bin/tp1_driver`, por integrar diversas rotinas Assembly do projeto. As evidências geradas foram:

- `evidence/raspberry/ex05/tp1_driver_objdump.txt`
- `evidence/raspberry/ex12/tp1_driver_symbols.txt`

Trechos relevantes:

```
4008a4: 9400017f bl 400ea0 <mp_add_192>
400978: 94000182 bl 400f80 <parse_i64>
400a24: 9400012d bl 400ed8 <u64_to_str>
400ad0: 94000160 bl 401050 <int_div_to_double>
400b90: 9400013e bl 401088 <parse_i64_range>
0000000000400c00 <main>:

0000000000400ea0 g      F .text 0000000000000038 mp_add_192
0000000000400ed8 g      F .text 00000000000000a8 u64_to_str
0000000000400f80 g      F .text 00000000000000c4 parse_i64
0000000000401050 g      F .text 0000000000000038 int_div_to_double
0000000000401088 g      F .text 000000000000005c parse_i64_range
0000000000400c00 g      F .text 00000000000000c4 main
```

Conclusão: a decomposição confirma a presença dos símbolos exigidos e a chamada efetiva dessas rotinas pelo `main` do driver final.

9 Exercício 6 — Soma multi-palavra de 192 bits

A rotina foi implementada em `src/asm/mp_add_192.s`. O contrato adotado foi:

- três words de 64 bits em memória, em ordem little-endian;
- `word0` como parte menos significativa;
- retorno do carry final em `x0`.

Evidência gerada:

- `evidence/raspberry/ex06/ex6_mp_add_192_output.txt`

Resumo da execução:

```
case=no-carry          => PASS
case=carry-low-mid     => PASS
case=carry-through-high => PASS
case=final-overflow    => PASS
STATUS FINAL: PASS
```

Conclusão: a rotina tratou corretamente tanto os casos sem carry quanto os cenários com propagação intermediária, propagação até a word mais significativa e overflow final de 192 bits.

10 Exercício 7 — Subtração multi-palavra de 128 bits

A rotina foi implementada em `src/asm/mp_sub_128.s`. O contrato adotado foi:

- duas words de 64 bits em memória, em ordem little-endian;
- `word0` como parte menos significativa;
- retorno de underflow/borrow final em `x0`, com 0 = sem borrow final e 1 = com borrow final.

Evidências:

- `evidence/raspberry/ex07/ex7_mp_sub_128_output.txt`
- `evidence/raspberry/ex07/gdb_mp_sub_128_borrow.txt`

Resumo da execução:

```
case=no-borrow => PASS
case=borrow    => PASS
STATUS FINAL: PASS
```

Conclusão: a rotina entregou resultados corretos tanto no caso sem borrow quanto no caso com borrow, e a sessão de GDB corroborou o comportamento das flags e do valor retornado.

11 Exercício 8 — Conversão de inteiro para string

A rotina foi implementada em `src/asm/u64_to_str.s`. O contrato definido foi:

- entrada: `uint64_t value`, ponteiro para buffer e tamanho do buffer;
- saída: string decimal ASCII terminada em `\0`;
- retorno: quantidade de caracteres sem contar o terminador nulo;
- política para buffer insuficiente: string vazia e retorno 0.

Tamanho mínimo garantido para qualquer `uint64_t`: 21 bytes.

Evidência:

- `evidence/raspberry/ex08/ex8_u64_to_str_output.txt`

Resumo:

```
case=zero          => PASS
case=one-digit     => PASS
case=many-digits  => PASS
case=uint64-max    => PASS
STATUS FINAL: PASS
```

Conclusão: a rotina produziu strings corretas para zero, números simples, números com múltiplos dígitos e `UINT64_MAX`, sempre com tamanho compatível com o conteúdo gerado.

12 Exercício 9 — Conversão de string para inteiro

A rotina foi implementada em `src/asm/parse_i64.s`. O contrato adotado foi:

- entrada: ponteiro para string ASCII;
- sinal opcional + ou - apenas no primeiro caractere;
- saída: valor convertido em memória e código de status em `w0`.

Estados definidos:

- 0: OK
- 1: EMPTY
- 2: INVALID
- 3: OVERFLOW

Evidência:

- `evidence/raspberry/ex09/ex9_parse_i64_output.txt`

Resumo:

```
"0"      => OK
"42"     => OK
"-7"     => OK
"+15"    => OK
"12abc"  => INVALID
""       => EMPTY
STATUS FINAL: PASS
```

Conclusão: a rotina identificou corretamente o fim da string, aceitou sinal apenas no início e rejeitou entradas inválidas sem marcar falso sucesso.

13 Exercício 10 — Conversão para double e divisão

A rotina foi implementada em `src/asm/int_div_to_double.s`. O contrato adotado foi:

- receber dois inteiros signed de 64 bits;
- converter ambos com `scvtf`;
- dividir com `fdiv`;
- gravar o resultado em `double *out`;
- retornar `DIV_BY_ZERO` e gravar `0.0` quando `b = 0`.

Evidência:

- `evidence/raspberry/ex10/ex10_int_div_to_double_output.txt`

Resumo:

```
7 / 2    => 3.5000000000000000 PASS
-9 / 4    => -2.2500000000000000 PASS
5 / 0     => DIV_BY_ZERO PASS
STATUS FINAL: PASS
```

Conclusão: os resultados numéricos observados no Raspberry confirmam a correção da conversão para ponto flutuante e da política de erro definida para divisor zero.

14 Exercício 11 — Validação de faixa em parsing numérico

A rotina foi implementada em `src/asm/parse_i64_range.s`. A faixa definida no código foi:

- MIN = -1000
- MAX = 1000

Política de status:

- 0: OK
- 2: INVALID
- 4: OUT_OF_RANGE

Evidência:

- `evidence/raspberry/ex11/ex11_parse_i64_range_output.txt`

Resumo:

```
"500"    => OK
"-1001"  => OUT_OF_RANGE
"1001"   => OUT_OF_RANGE
"9x"     => INVALID
""       => INVALID
"-10"    => OK
STATUS FINAL: PASS
```

Conclusão: a rotina distinguiu adequadamente erros de formato de violações de faixa, mantendo uma política de erro verificável por execução.

15 Exercício 12 — Integração final

O executável final de integração está em `build/bin/tp1_driver`, construído a partir de `src/c/tp1_driver.c`. Ele integra diretamente as rotinas:

- `mp_add_192`
- `parse_i64`
- `u64_to_str`
- `int_div_to_double`
- `parse_i64_range`

Evidências:

- `evidence/raspberry/ex12/ex12_driver_output.txt`
- `evidence/raspberry/ex12/tp1_driver_symbols.txt`
- `evidence/raspberry/ex12/full_test_suite_output.txt`
- `evidence/raspberry/ex05/tp1_driver_objdump.txt`

Trecho da execução:


```
[driver] mp_add_192 =>
↳ 0x00000000000000001_0000000000000000_0000000000000000 carry=0 [PASS]
[driver] parse_i64 => status=OK value=-128 [PASS]
[driver] u64_to_str => text="20260426" len=8 [PASS]
[driver] int_div => status=OK result=3.500000 [PASS]
[driver] parse_range => status=OUT_OF_RANGE [PASS]
STATUS FINAL DRIVER: PASS
```

Conclusão: o pipeline final compila, executa no Raspberry, é auditável por `objdump` e depurável por GDB, atendendo ao objetivo de entrega reproduzível solicitado no enunciado.

16 Consolidação das Evidências

Todas as evidências finais executadas no hardware real estão em `evidence/raspberry/`. Os principais artefatos são:

- `ex02/ex2_soc_addrs_output.txt`
- `ex02/ex2_soc_addrs_objdump.txt`
- `ex03/make_clean.txt`
- `ex03/make_build.txt`
- `ex04/gdb_mp_sub_128_borrow.txt`
- `ex05/tp1_driver_objdump.txt`
- `ex06/ex6_mp_add_192_output.txt`
- `ex07/ex7_mp_sub_128_output.txt`
- `ex07/gdb_mp_sub_128_borrow.txt`
- `ex08/ex8_u64_to_str_output.txt`
- `ex09/ex9_parse_i64_output.txt`
- `ex10/ex10_int_div_to_double_output.txt`
- `ex11/ex11_parse_i64_range_output.txt`
- `ex12/ex12_driver_output.txt`
- `ex12/tp1_driver_symbols.txt`
- `ex12/full_test_suite_output.txt`

17 Conclusão Final

O trabalho entregou:

- infraestrutura de projeto compatível com o Raspberry Pi Zero 2W;
- automação de compilação e execução por `Makefile`;
- rotinas Assembly ARM64 cobrindo aritmética multi-palavra, conversão numérica, parsing e validação de faixa;
- validação preliminar com QEMU e validação final no hardware real;
- relatório técnico com trechos objetivos de execução, `objdump` e GDB.

Com isso, os objetivos técnicos do TP1 foram atendidos de forma reproduzível e documentada.